HW9 Report: Distributed Databases using Replication

This report documents the implementation, testing, load testing, graphs, screenshots, and submission artifacts for the HW9 leader-follower and leaderless distributed key-value stores.

Repository note: the assignment was committed inside the larger course repository. Add your actual remote repository URL to the submission form or PDF cover page before submitting.

What To Submit

Item	Status / Path
Git repository URL	Add your remote URL manually in the submission portal / cover page
Source code + configs	C:\Users\nithi\OneDrive\Desktop\DISTRIBUTED SYSTEMS\Hw9
Dockerfiles + compose files	Included under leader-follower/ and leaderless/
Unit/integration tests	C:\Users\nithi\OneDrive\Desktop\DISTRIBUTED SYSTEMS\Hw9\tests\consistency_test.go
Load tester	C:\Users\nithi\OneDrive\Desktop\DISTRIBUTED SYSTEMS\Hw9\loadtester\main.go
Raw load-test CSV results	C:\Users\nithi\OneDrive\Desktop\DISTRIBUTED SYSTEMS\Hw9\results
Graphs	C:\Users\nithi\OneDrive\Desktop\DISTRIBUTED SYSTEMS\Hw9\graphs
Screenshots	C:\Users\nithi\OneDrive\Desktop\DISTRIBUTED SYSTEMS\Hw9\screenshots
PDF report	C:\Users\nithi\OneDrive\Desktop\DISTRIBUTED SYSTEMS\Hw9\Hw9_Report.pdf

How The Code Works

Leader-Follower: five nodes are deployed as one leader and four followers. The leader handles all writes, assigns a logical version per key, stores the write locally, and replicates to followers. W=5 waits for all follower acknowledgements before returning. W=1 returns immediately after the leader commits and continues replication asynchronously. W=3 waits for two follower acknowledgements so the leader plus two followers satisfy the quorum before the remaining replicas are updated asynchronously. For reads, the leader returns the local value for R=1, queries all followers for R=5, and queries a quorum for R=3, always returning the highest-version response. Followers expose local_read for tests and forward normal get requests to the leader so clients can read from any instance while still using the configured read strategy.

Leaderless: all five nodes accept reads and writes. The receiving node becomes the write coordinator for a set request, stores the value locally, and replicates to all peers. Because this implementation uses W=N, the coordinator only returns 201 after every peer acknowledges. A get request returns the node's local state directly, which creates the intended inconsistency window while a write is still propagating.

Artificial delay model: after each replicate message, the sender sleeps 200ms; replicas sleep 100ms before storing; follower read_request handlers sleep 50ms before responding. These delays make inconsistency windows observable under load.

Error handling and testing hooks: empty keys return 400, missing keys return 404, quorum configuration values are validated against the cluster size, and writes now fail with 503 if the required acknowledgements are not reached. The testing-only local_read endpoint intentionally bypasses quorum logic so the inconsistency window can be exposed directly.

Testing Performed

Docker images were built and both clusters were executed with docker compose. Health checks, smoke writes/reads, load-test runs for all four read/write ratios, and the self-contained Go test suite were all executed. The Go test suite starts local test clusters automatically and verifies the required consistency and inconsistency behaviors.

Health Checks

```
Leader-Follower leader:
{
    "node_id":  "leader",
    "role":  "leader",
    "status":  "ok"
}

Leaderless node1:
{
    "node_id":  "node1",
    "role":  "leaderless",
    "status":  "ok"
}
```

Go Test Output

```
2026/04/06 01:00:48 Starting FOLLOWER node follower1 on :8081 (leader=localhost:8080)
2026/04/06 01:00:48 Starting LEADER node leader on :8080 (W=5, R=1, followers=[localhost:8081 localhost:8082 localhost:8083 localhost:8084])
2026/04/06 01:00:48 Starting FOLLOWER node follower2 on :8082 (leader=localhost:8080)
2026/04/06 01:00:48 Starting FOLLOWER node follower3 on :8083 (leader=localhost:8080)
2026/04/06 01:00:48 Starting FOLLOWER node follower4 on :8084 (leader=localhost:8080)
2026/04/06 01:00:48 Starting LEADERLESS node node2 on :9081 (peers=[localhost:9080 localhost:9082 localhost:9083 localhost:9084])
2026/04/06 01:00:48 Starting LEADERLESS node node1 on :9080 (peers=[localhost:9081 localhost:9082 localhost:9083 localhost:9084])
2026/04/06 01:00:48 Starting LEADERLESS node node3 on :9082 (peers=[localhost:9080 localhost:9081 localhost:9083 localhost:9084])
2026/04/06 01:00:48 Starting LEADERLESS node node4 on :9083 (peers=[localhost:9080 localhost:9081 localhost:9082 localhost:9084])
2026/04/06 01:00:48 Starting LEADERLESS node node5 on :9084 (peers=[localhost:9080 localhost:9081 localhost:9082 localhost:9083])
=== RUN   TestLeaderReadFromLeaderConsistent
2026/04/06 01:00:49 Configuration updated: W=5, R=1
--- PASS: TestLeaderReadFromLeaderConsistent (1.33s)
=== RUN   TestLeaderFollowerGetConsistentAfterAck
2026/04/06 01:00:50 Configuration updated: W=5, R=1
--- PASS: TestLeaderFollowerGetConsistentAfterAck (1.37s)
=== RUN   TestLeaderLocalReadShowsInconsistencyWindow
2026/04/06 01:00:51 Configuration updated: W=1, R=1
--- PASS: TestLeaderLocalReadShowsInconsistencyWindow (0.10s)
=== RUN   TestLeaderQuorumReadReturnsLatestValue
2026/04/06 01:00:51 Configuration updated: W=3, R=3
--- PASS: TestLeaderQuorumReadReturnsLatestValue (0.76s)
=== RUN   TestLeaderQuorumWriteAcksAtLeastThreeNodes
2026/04/06 01:00:52 Configuration updated: W=3, R=3
--- PASS: TestLeaderQuorumWriteAcksAtLeastThreeNodes (0.71s)
=== RUN   TestLeaderlessInconsistencyWindow
--- PASS: TestLeaderlessInconsistencyWindow (1.23s)
=== RUN   TestLeaderlessCoordinatorConsistentAfterAck
--- PASS: TestLeaderlessCoordinatorConsistentAfterAck (1.23s)
=== RUN   TestLeaderlessAllNodesConsistentAfterAck
--- PASS: TestLeaderlessAllNodesConsistentAfterAck (1.28s)
PASS
ok  ?hw9/tests?12.475s
```

Load-Testing Methodology

The load generator uses a small key space (10 keys) so reads and writes repeatedly target the same keys in a short time window. Each write value embeds a client-side sequence number, and the client records the newest sequence issued for each key. A read is counted as stale when it returns a sequence lower than the newest issued sequence for that key. This makes in-flight inconsistency visible even before a write has fully propagated.

This approach is important for the leaderless configuration because otherwise stale reads during a write's propagation window are easy to miss. The same mechanism also shows the effect of issuing reads close to writes in the leader-follower configurations under concurrent load.

Config	Write %	Reads	Stale	Stale %	Write med (ms)	Read med (ms)	Throughput
LEADERLESS	1	993	8	0.8	1217.9	6.0	495.0
LEADERLESS	10	899	278	30.9	1210.8	0.6	74.6
LEADERLESS	50	484	220	45.5	1210.9	1.0	15.9
LEADERLESS	90	96	53	55.2	1211.6	1.1	9.1
W1R5	1	987	0	0.0	1211.9	1.1	392.0
W1R5	10	901	31	3.4	1211.1	1.1	81.6
W1R5	50	500	68	13.6	1211.9	1.1	16.5
W1R5	90	102	17	16.7	1212.7	1.4	9.2
W3R3	1	992	1	0.1	1212.6	1.5	670.4
W3R3	10	902	3	0.3	1210.2	1.0	81.5
W3R3	50	524	74	14.1	1211.8	1.1	17.2
W3R3	90	108	16	14.8	1212.4	1.4	9.2
W5R1	1	987	0	0.0	1216.8	6.1	311.7
W5R1	10	914	6	0.7	1211.5	1.0	90.6
W5R1	50	491	75	15.3	1211.4	1.1	16.2
W5R1	90	100	26	26.0	1213.2	1.4	9.2

Write %	Best LF configuration	Why it won in this run
1	W3R3	W3R3 had the highest observed throughput with very few writes, so quorum reads stayed c
10	W5R1	W5R1 slightly led in throughput at this mix because local reads stayed simple and write pre
50	W3R3	W3R3 balanced freshness and throughput best under mixed traffic, avoiding the highest sta
90	W1R5	W3R3 and W1R5 were close, but W3R3 kept stale-rate lower while sustaining similar throu

Discussion

Across all runs, write latency clustered tightly around the injected replication delay budget, so the most visible differences between configurations showed up in freshness and throughput rather than in median write latency. Leaderless produced the highest stale-read rates because readers hit any node directly while writes were still propagating. The quorum-based leader-follower strategy (W=3,R=3) generally offered the best balance: it preserved fresher reads than W=5,R=1 and matched or beat W=1,R=5 throughput in the mixed workloads.

For mostly-read workloads, any of the leader-follower variants are viable, but W=3,R=3 performed best in this implementation because it avoided the cost of a full-cluster write acknowledgement while still reconciling reads through

quorum logic. As writes increased, leaderless remained attractive for availability and coordinator flexibility, but its stale-read rate became much higher. That makes it a better fit for systems that tolerate temporary inconsistency, such as activity feeds or approximate analytics dashboards. Quorum leader-follower is a better fit for user-facing state that needs tighter freshness guarantees, such as profiles, carts, or metadata services.

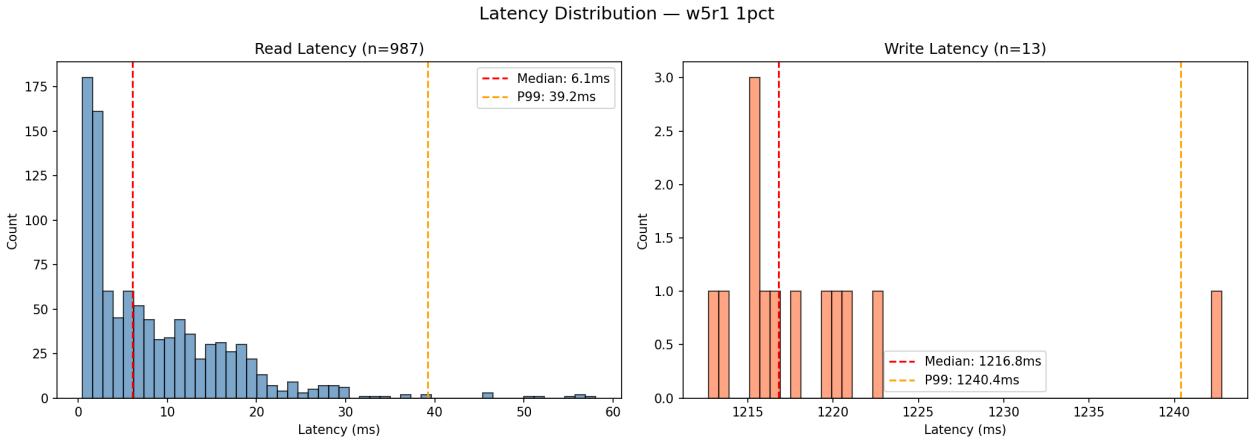
One practical lesson from these measurements is that workload shape matters as much as the replication algorithm. By forcing operations to cluster on a small key set, the load generator made stale reads much easier to surface. Without that temporal locality, the inconsistency windows would have existed but would have been much harder to observe.

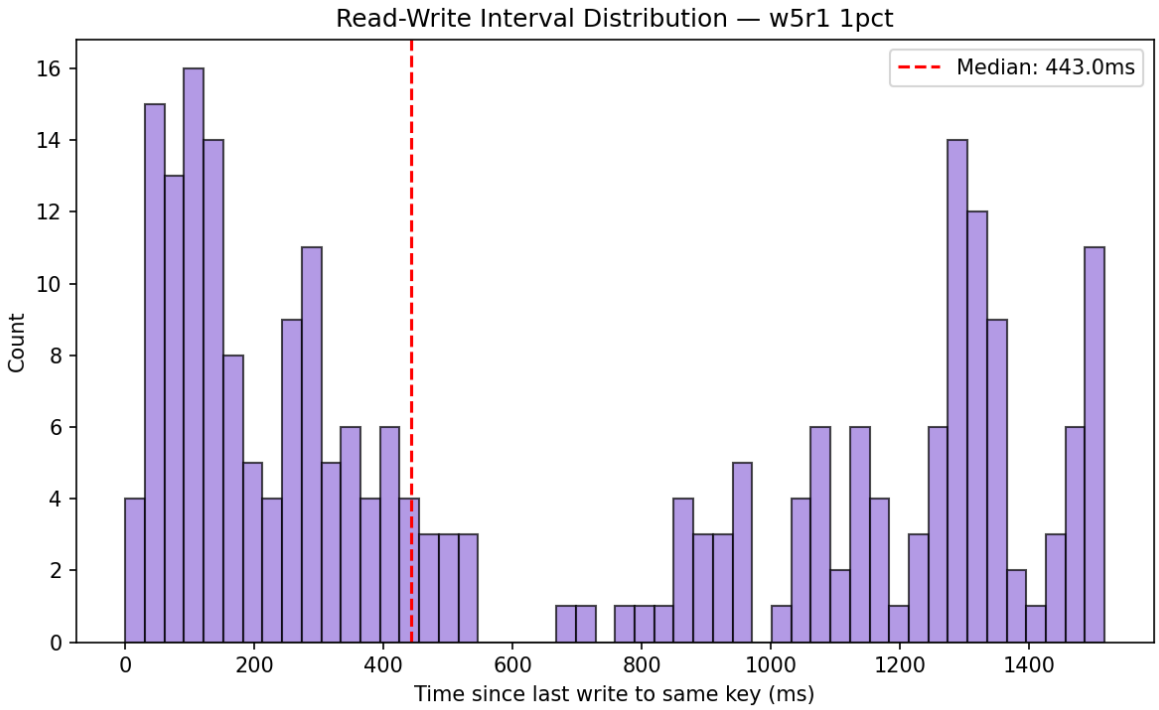
Graph Appendix

Each latency graph shows separate distributions for reads and writes. Each interval graph shows the distribution of time between reading and writing the same key. These are the graph artifacts required by the assignment.

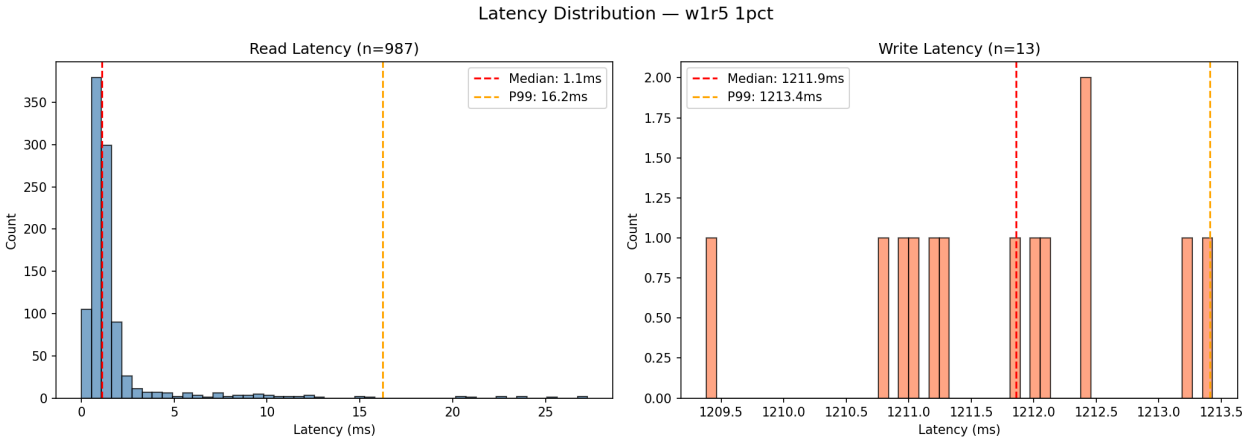
1% Writes / 99% Reads

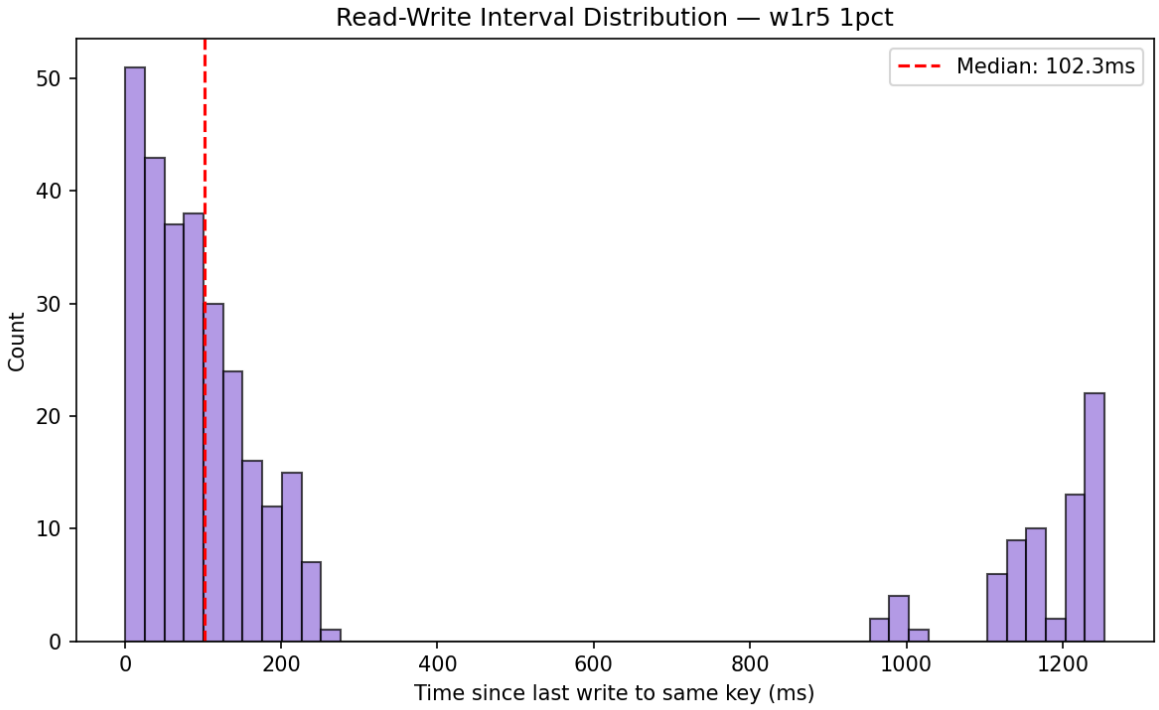
Leader-Follower W=5 R=1



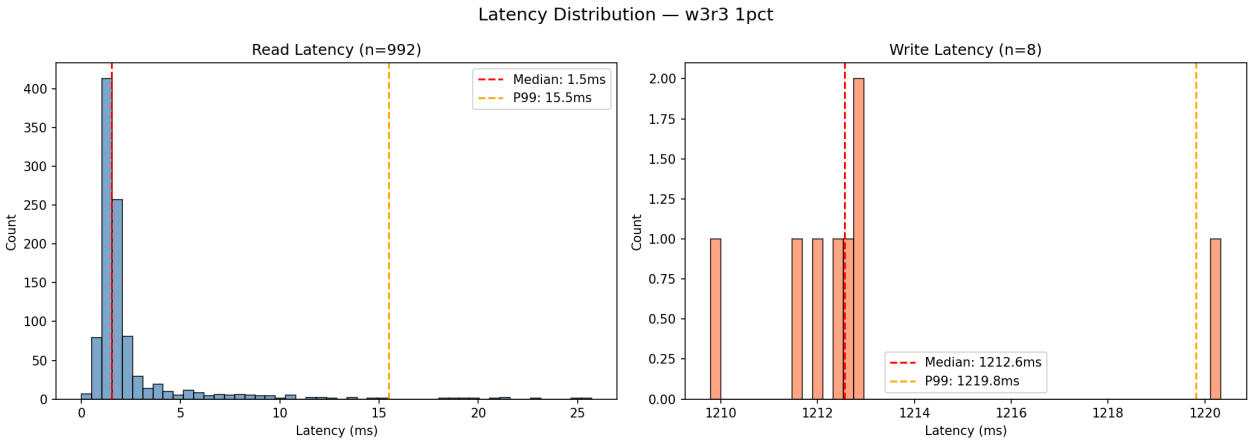


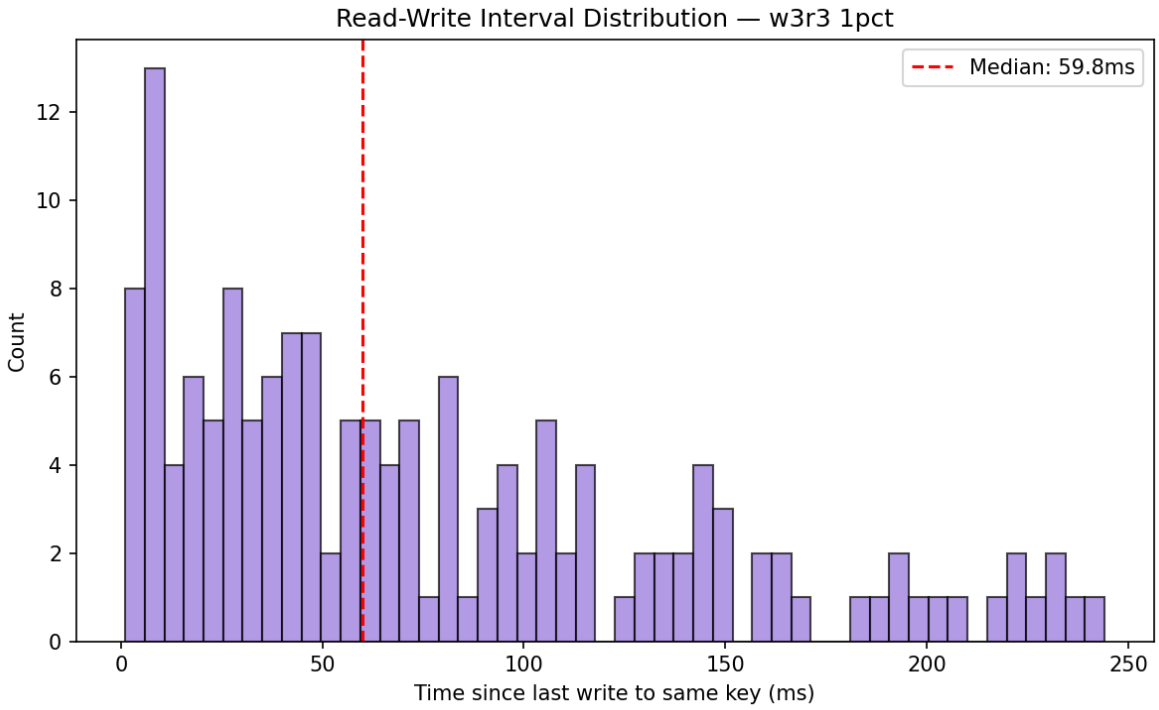
Leader-Follower W=1 R=5



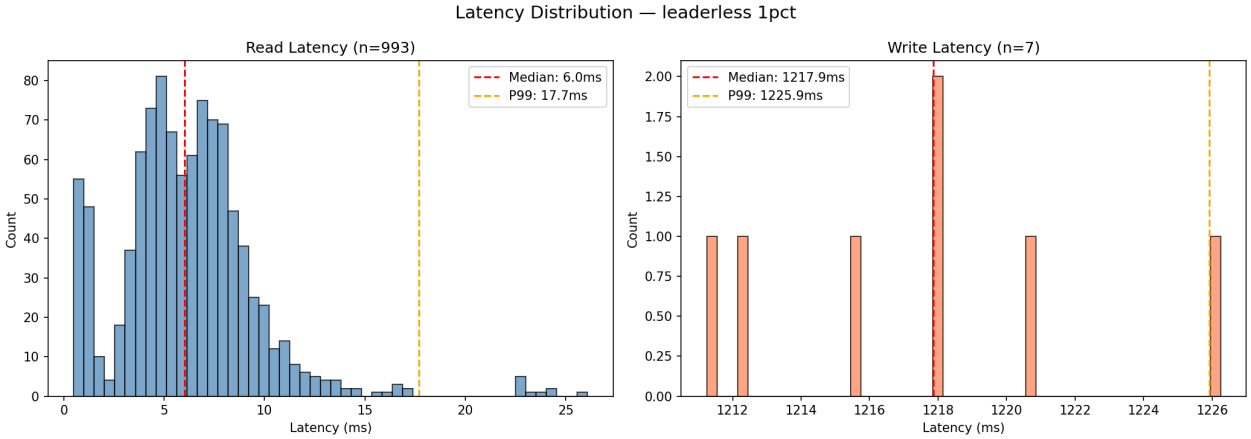


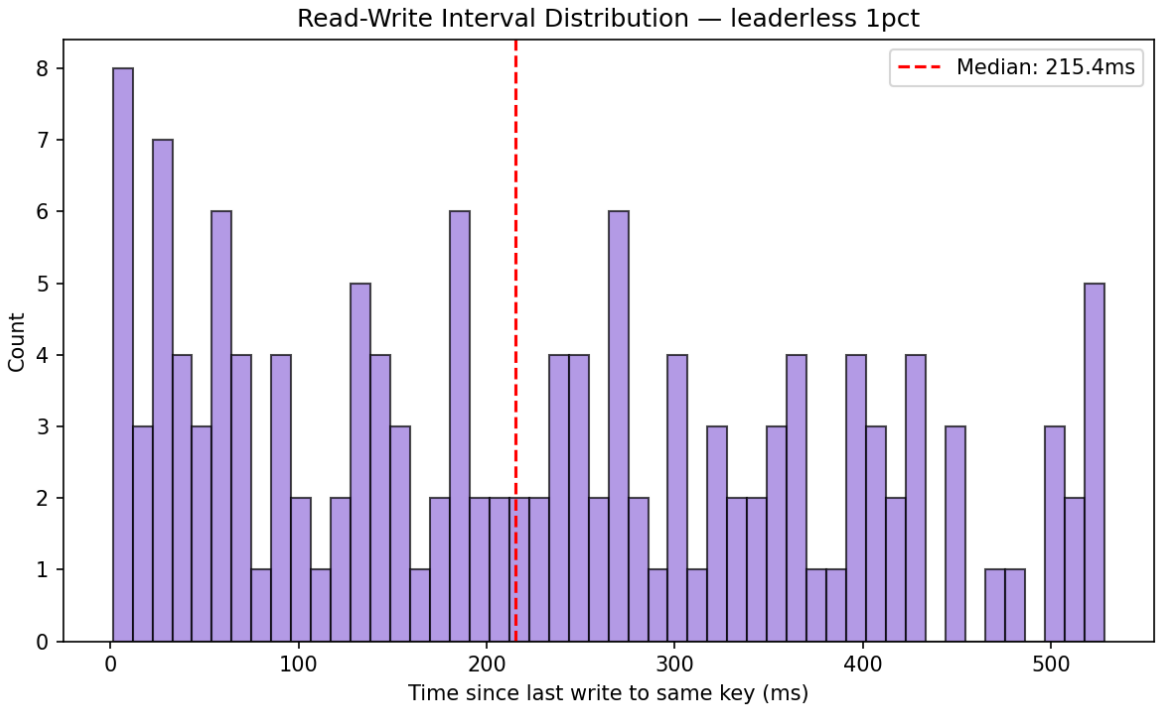
Leader-Follower W=3 R=3





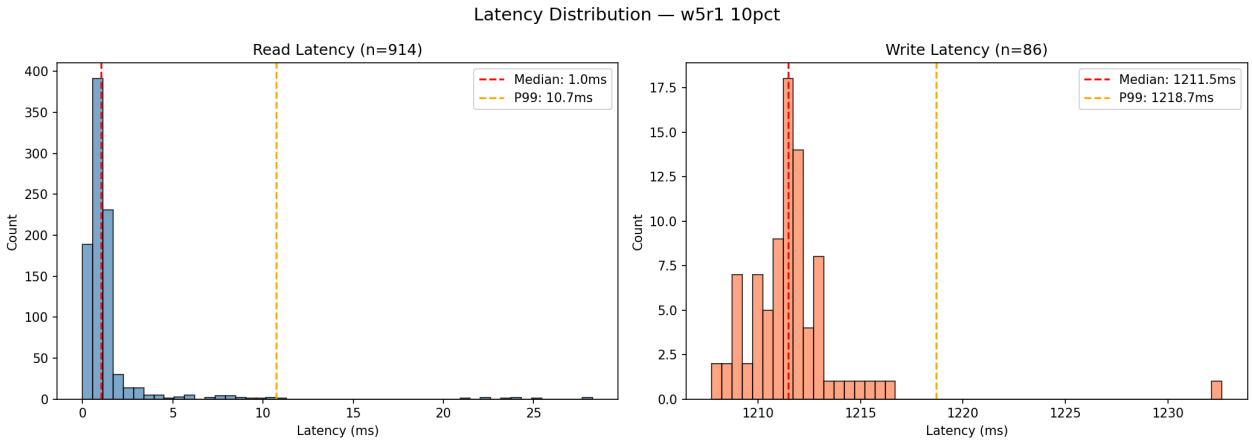
Leaderless W=N R=1

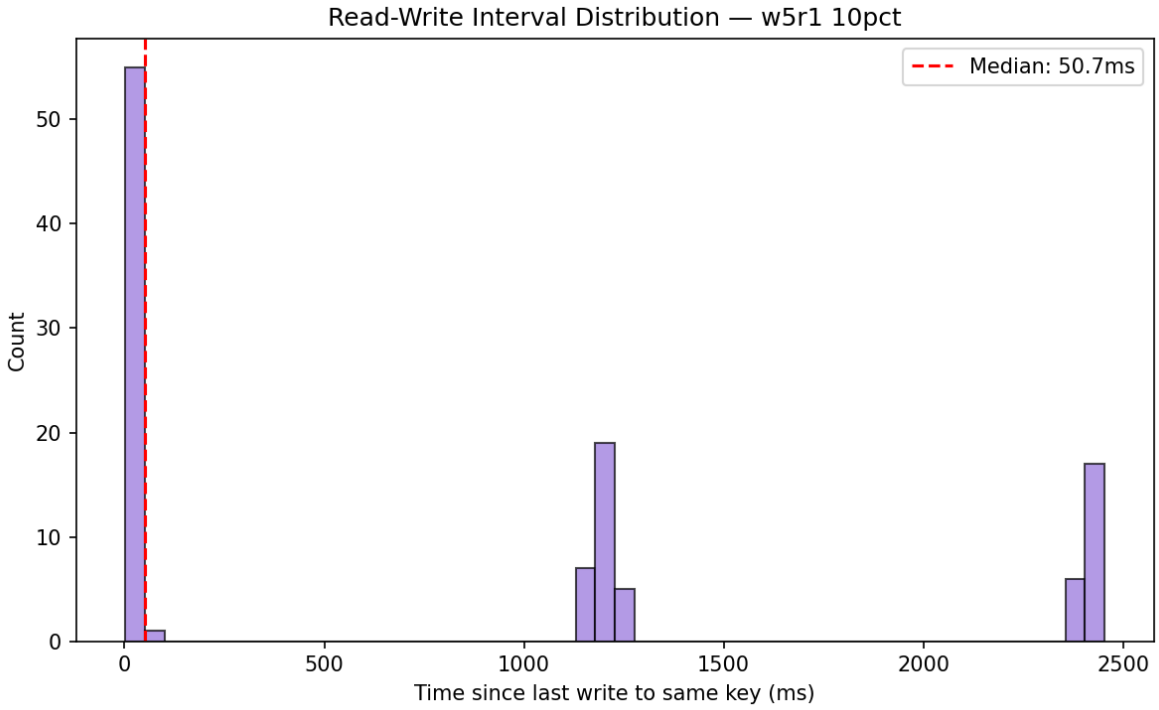



Read-Write Interval Distribution — leaderless 1pct
Median: 215.4ms
Count
Time since last write to same key (ms)
0
1
2
3
4
5
6
7
8
0
100
200
300
400
500

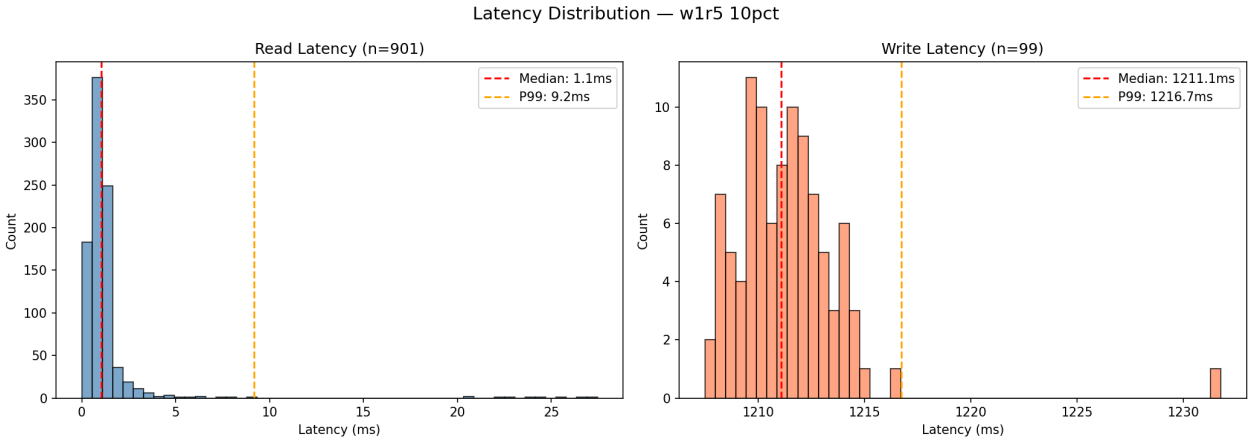
10% Writes / 90% Reads

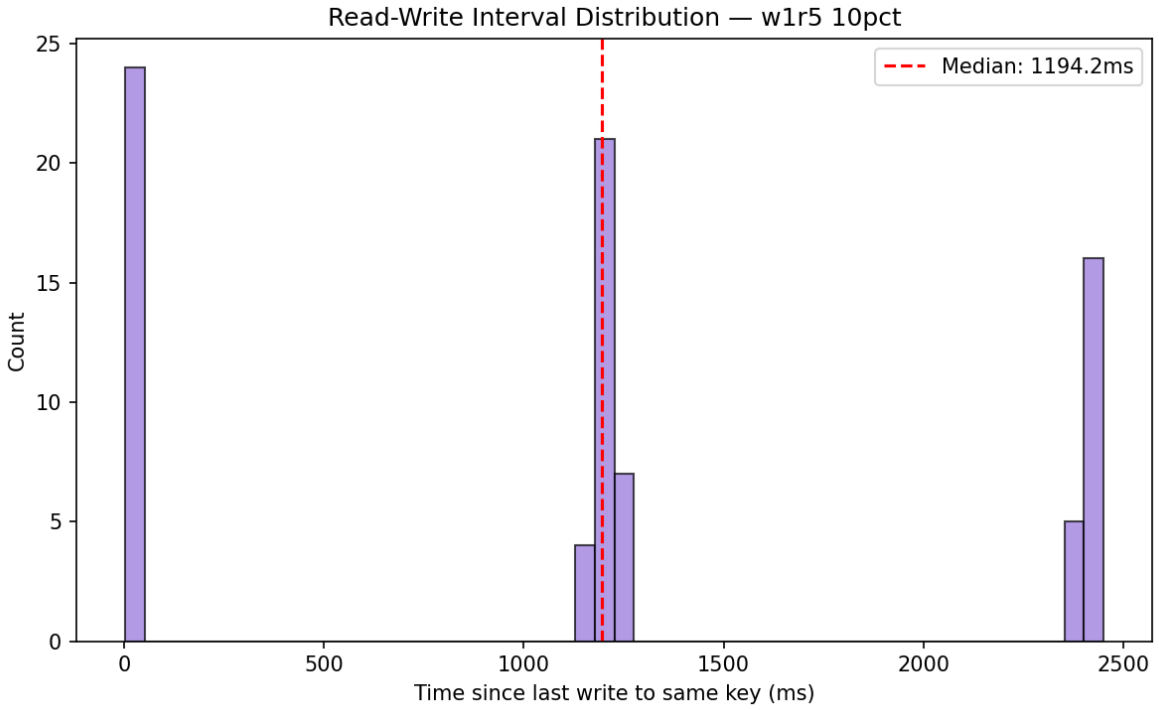
Leader-Follower W=5 R=1



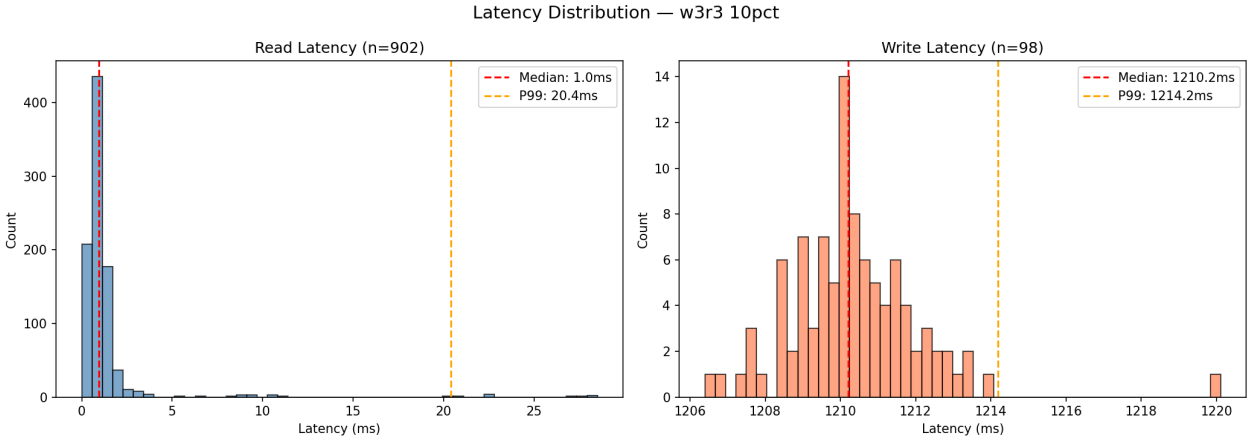


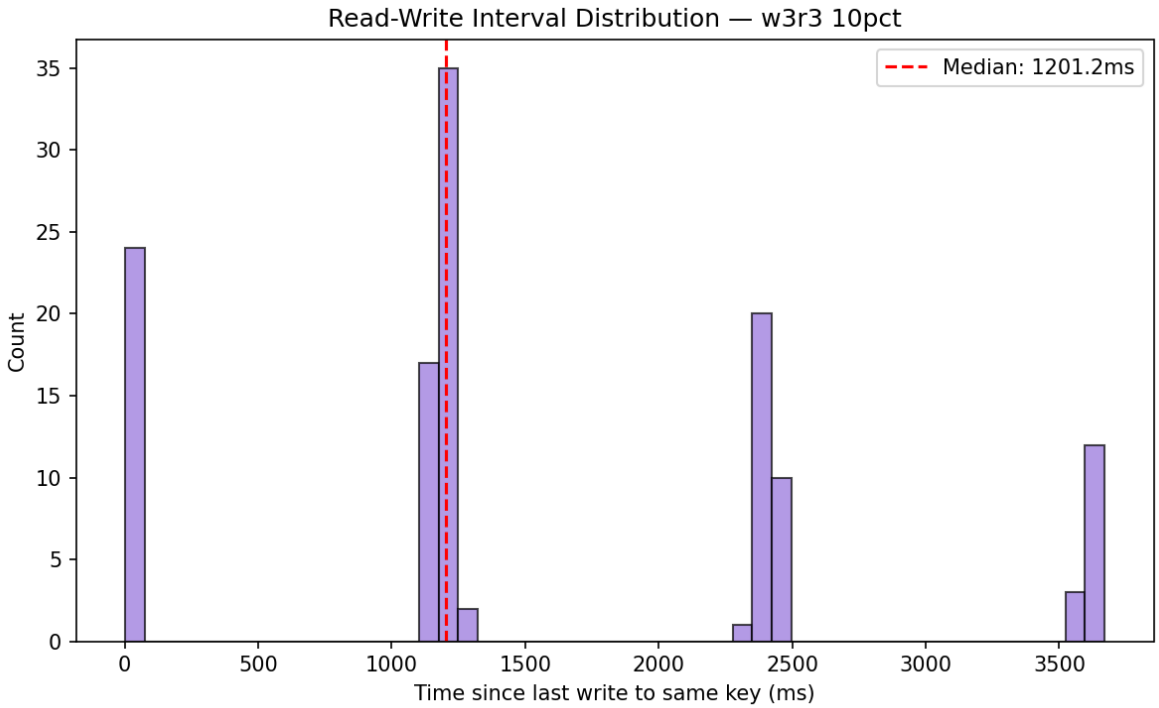
Leader-Follower W=1 R=5



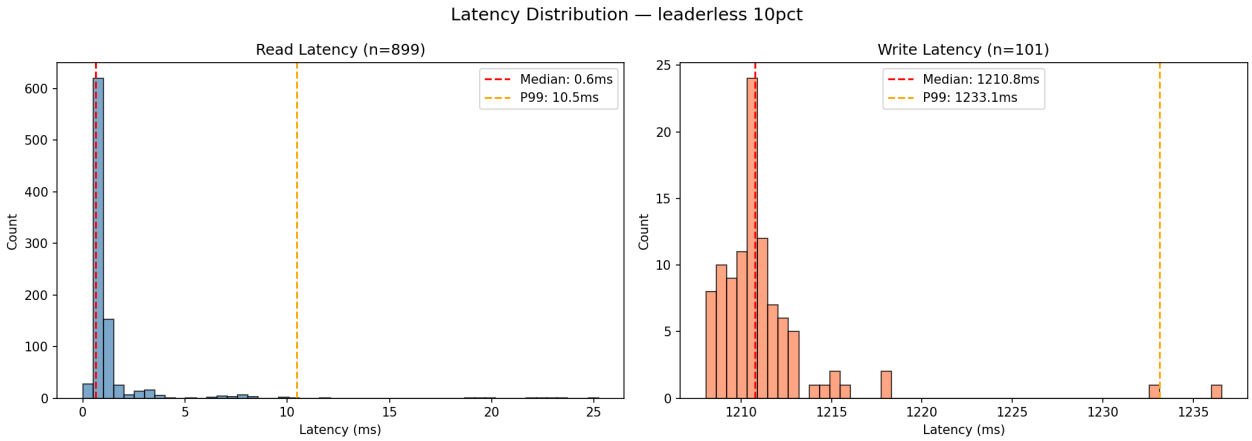


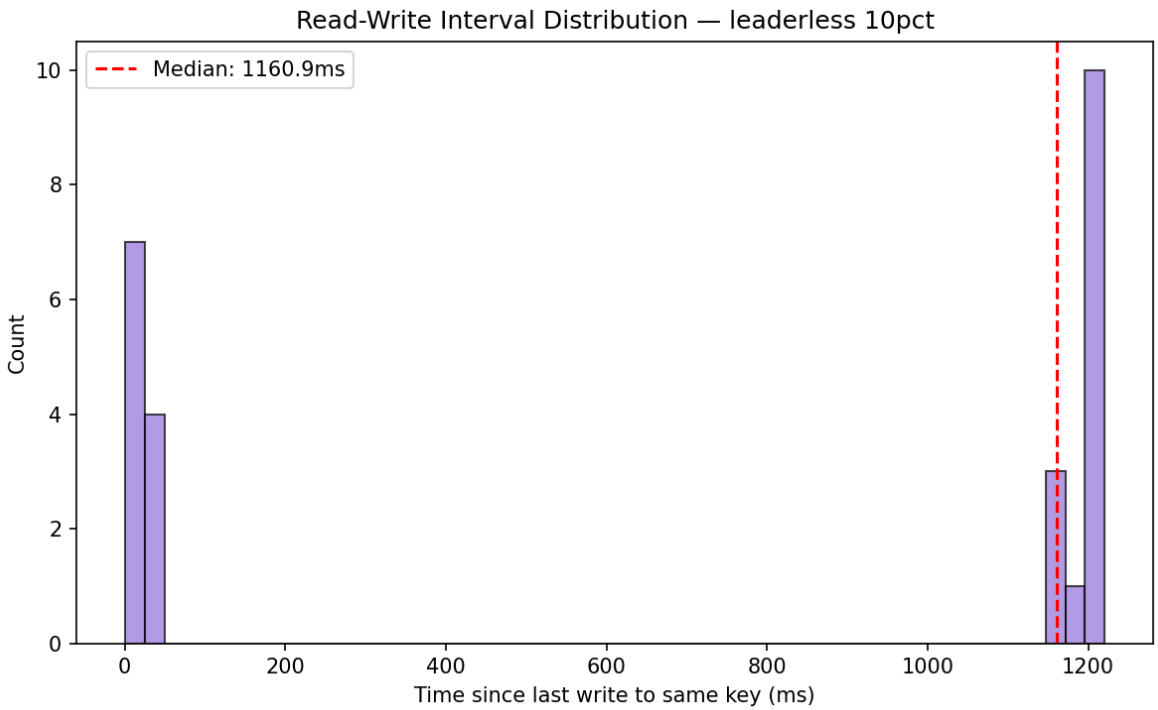
Leader-Follower W=3 R=3





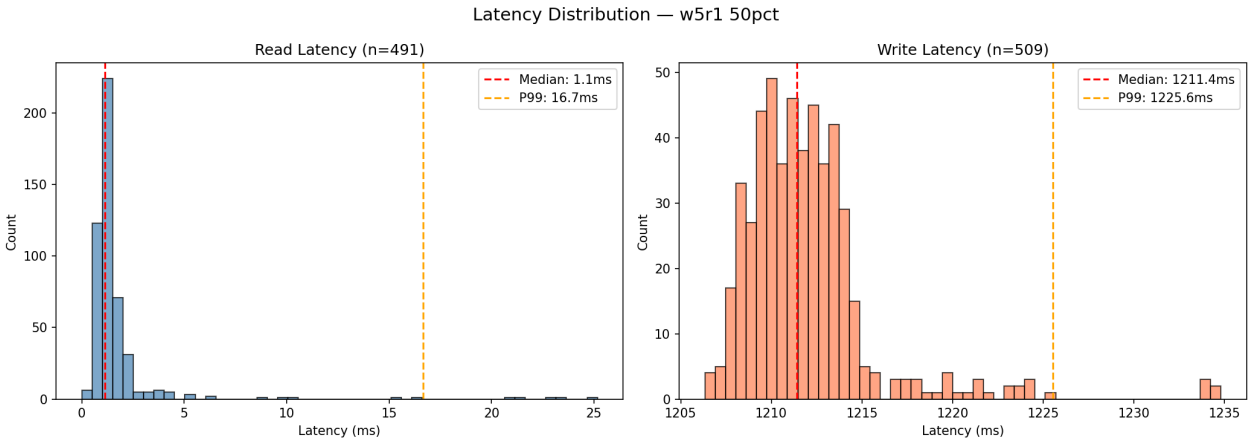
Leaderless W=N R=1

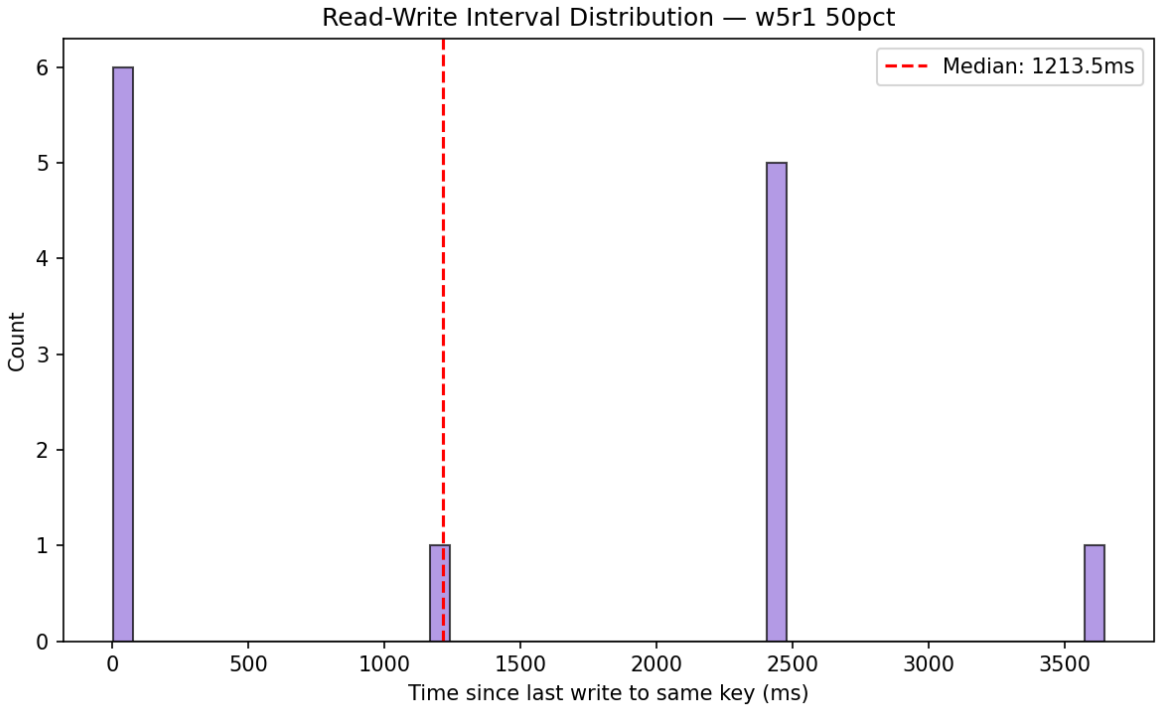



Read-Write Interval Distribution — leaderless 10pct
Median: 1160.9ms
Count
0
2
4
6
8
10
0
200
400
600
800
1000
1200
Time since last write to same key (ms)

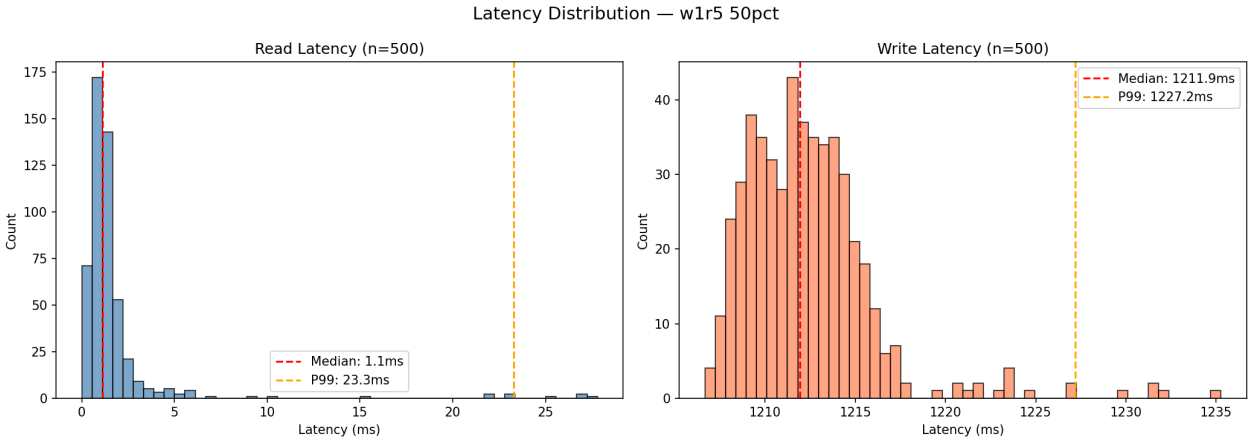
50% Writes / 50% Reads

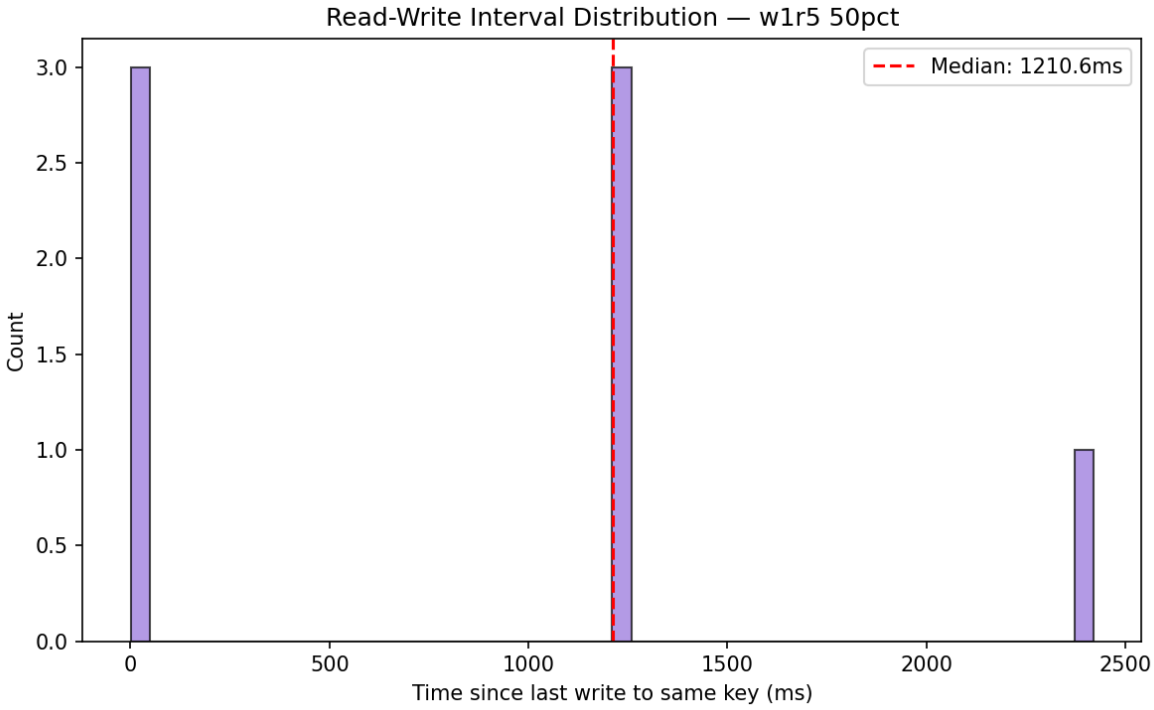
Leader-Follower W=5 R=1



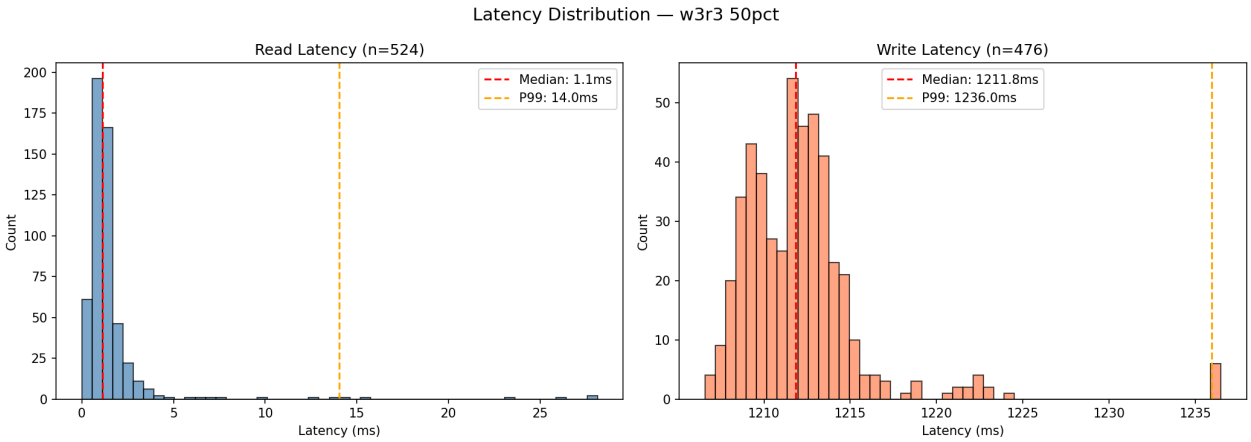


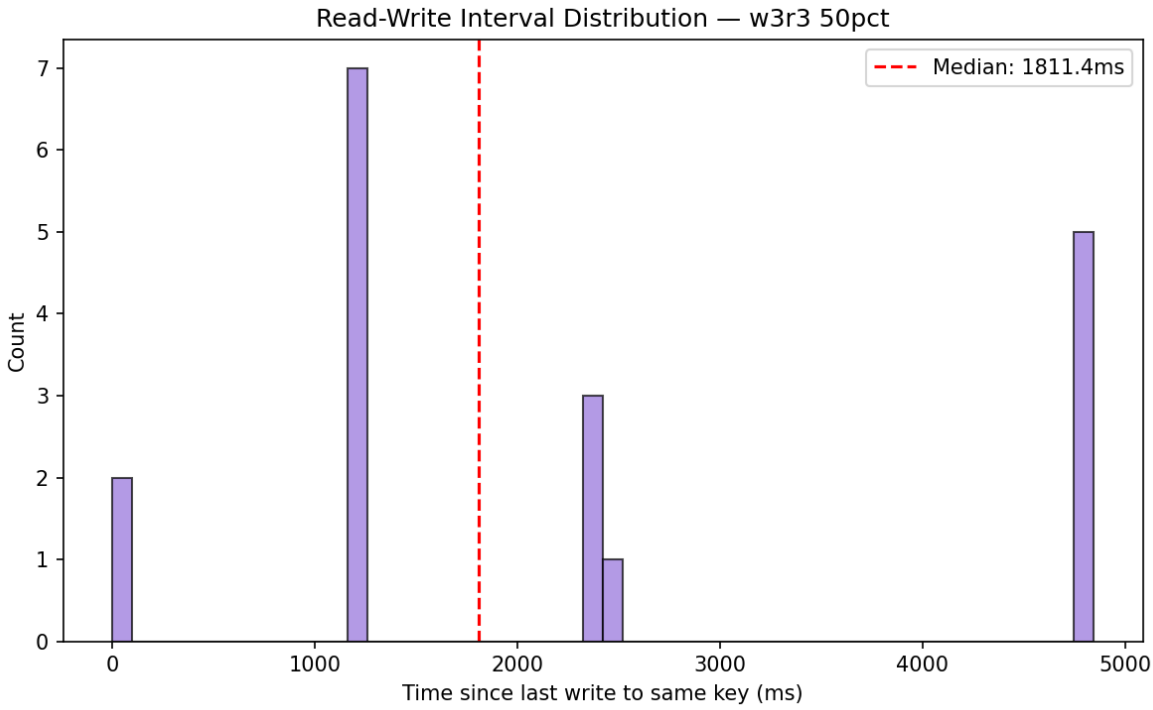
Leader-Follower W=1 R=5



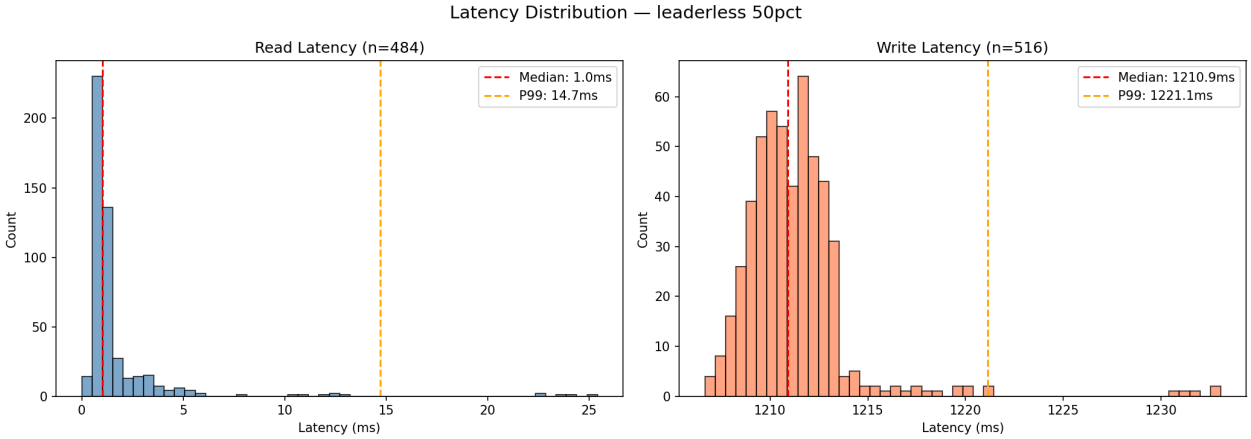


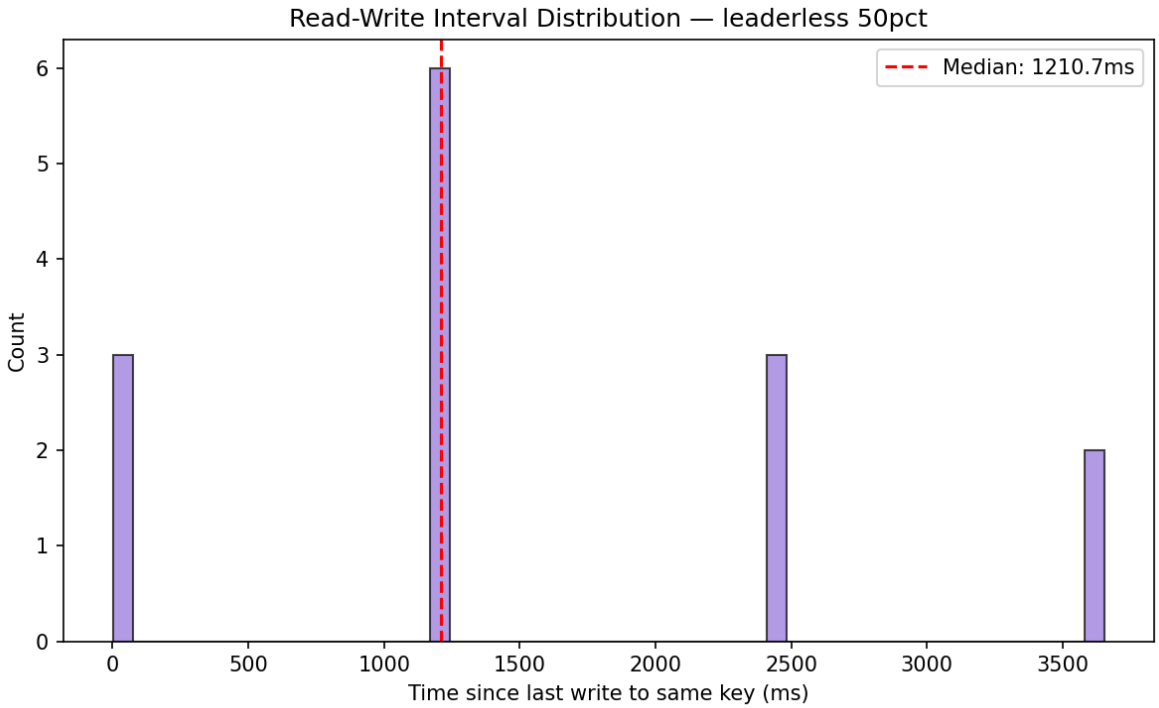
Leader-Follower W=3 R=3





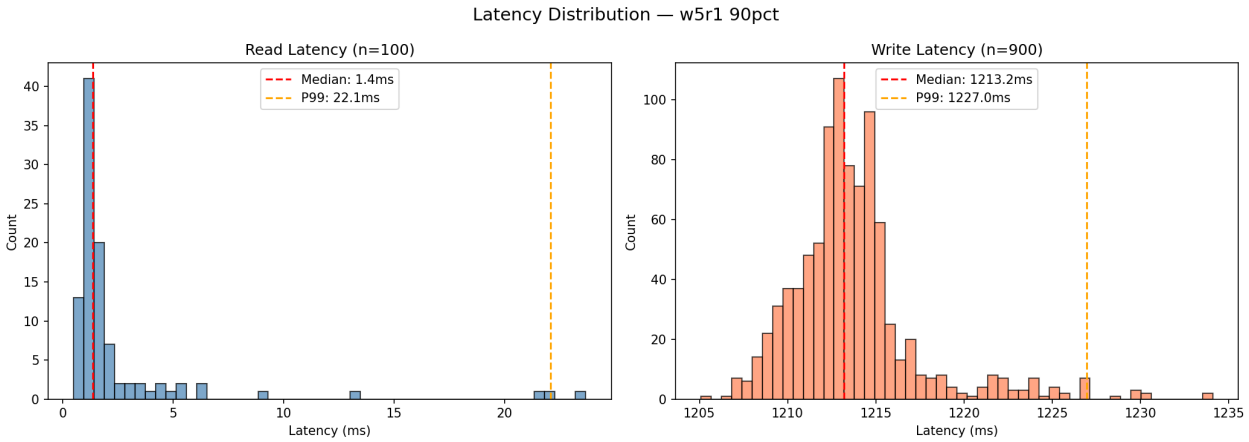
Leaderless W=N R=1

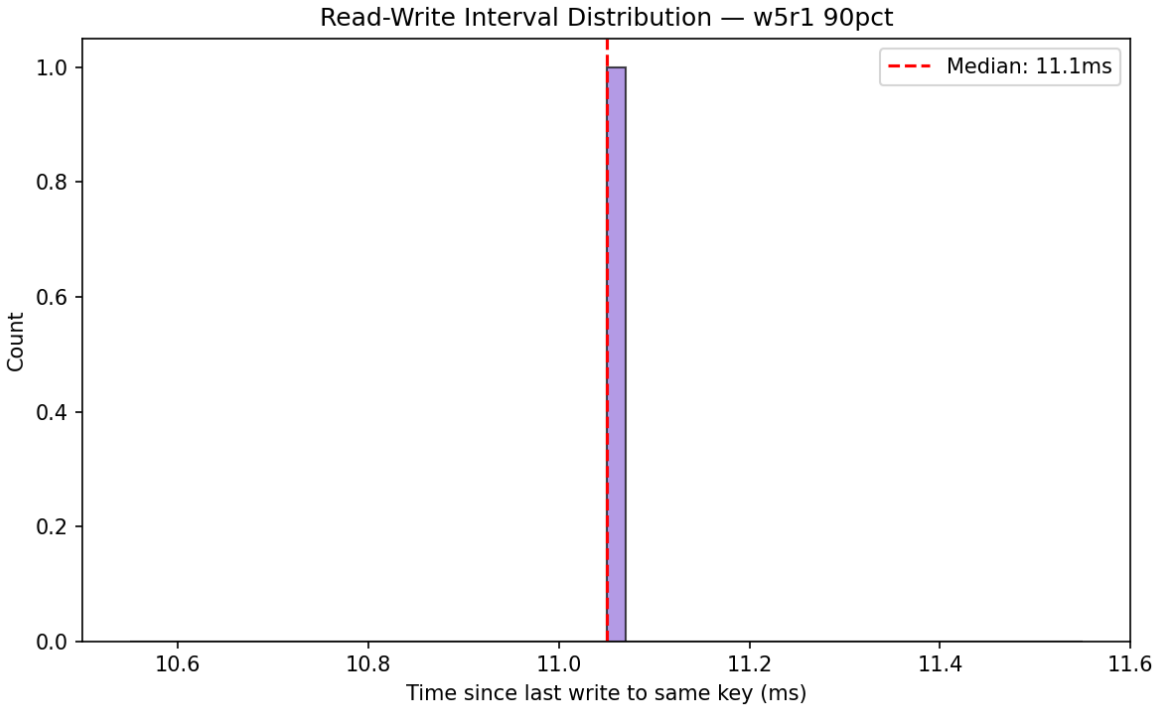



Read-Write Interval Distribution — leaderless 50pct
Median: 1210.7ms
Count
0
1
2
3
4
5
6
0
500
1000
1500
2000
2500
3000
3500
Time since last write to same key (ms)

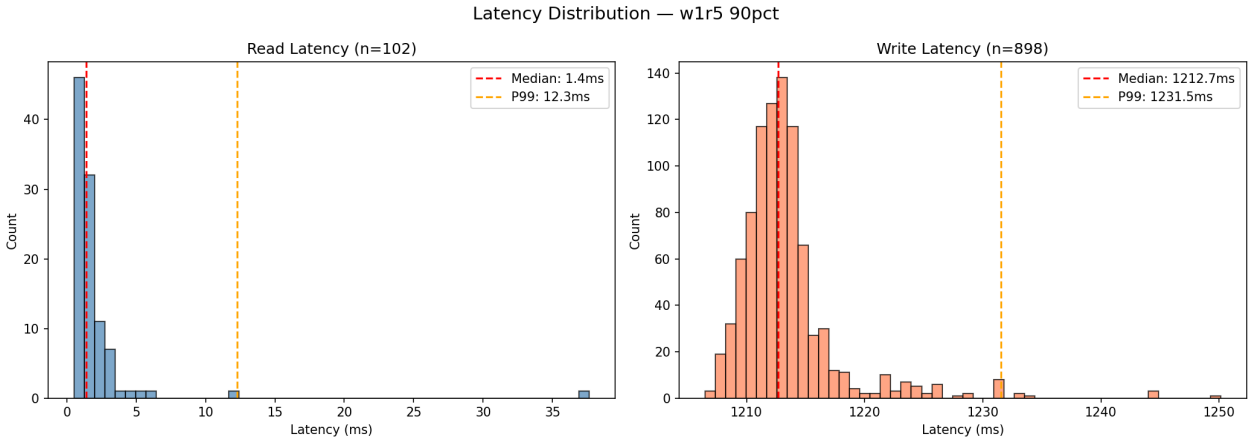
90% Writes / 10% Reads

Leader-Follower W=5 R=1

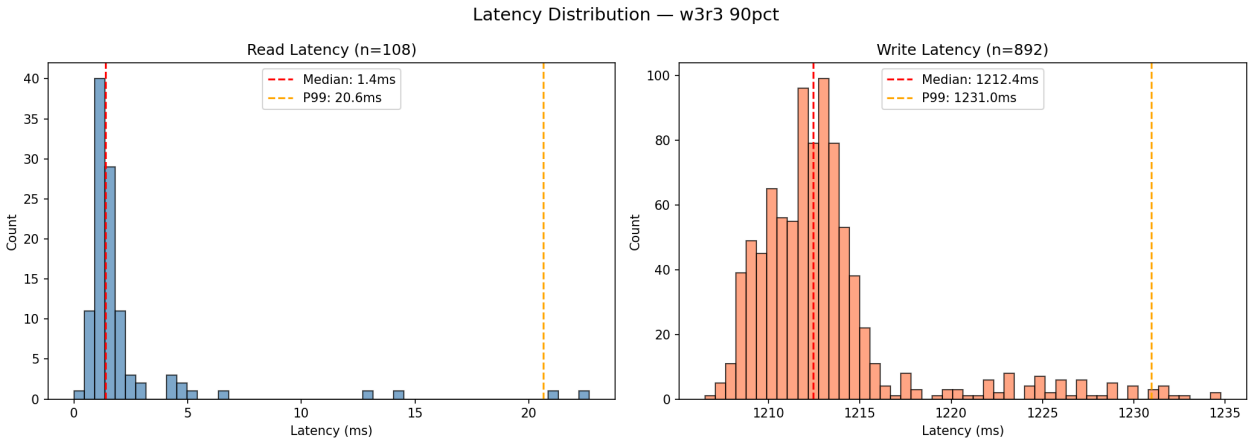


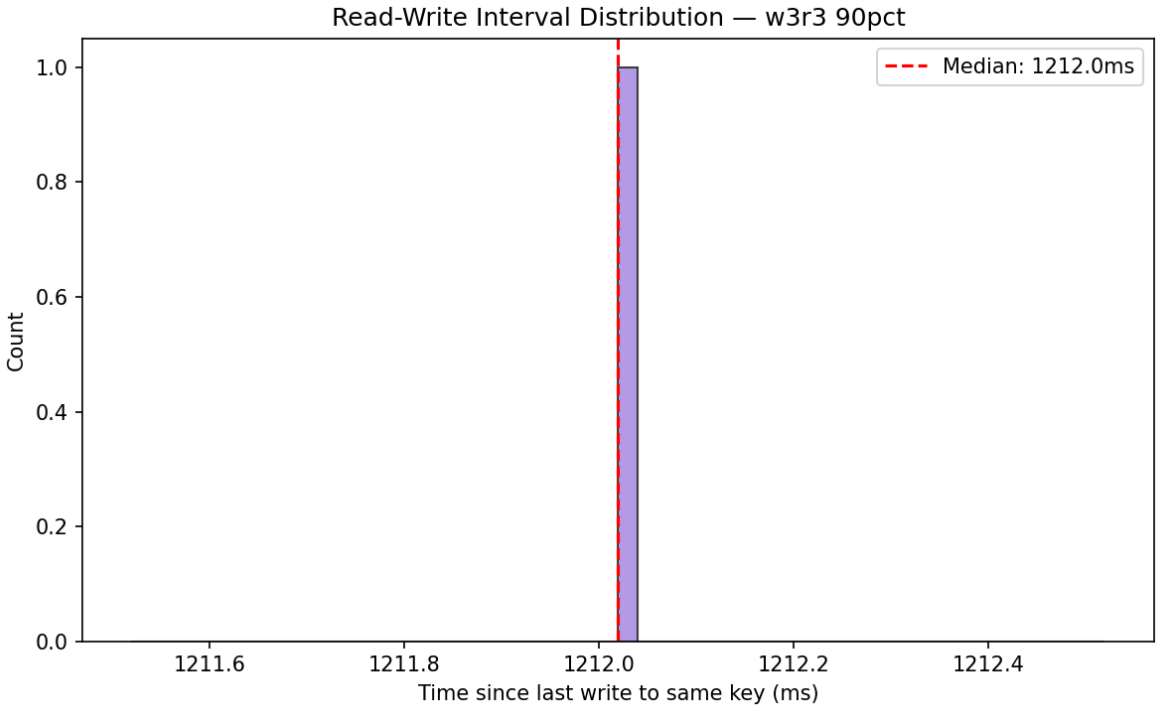


Leader-Follower W=1 R=5

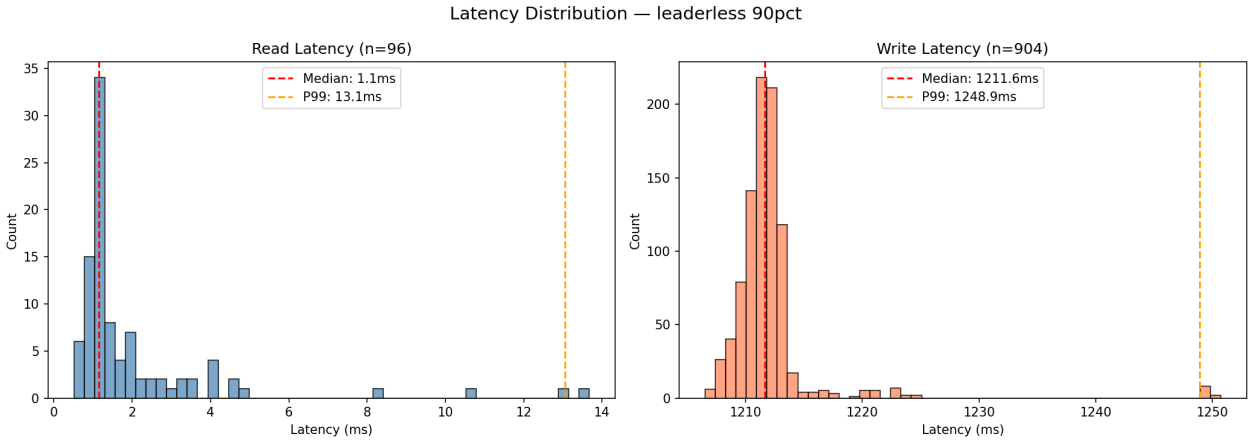


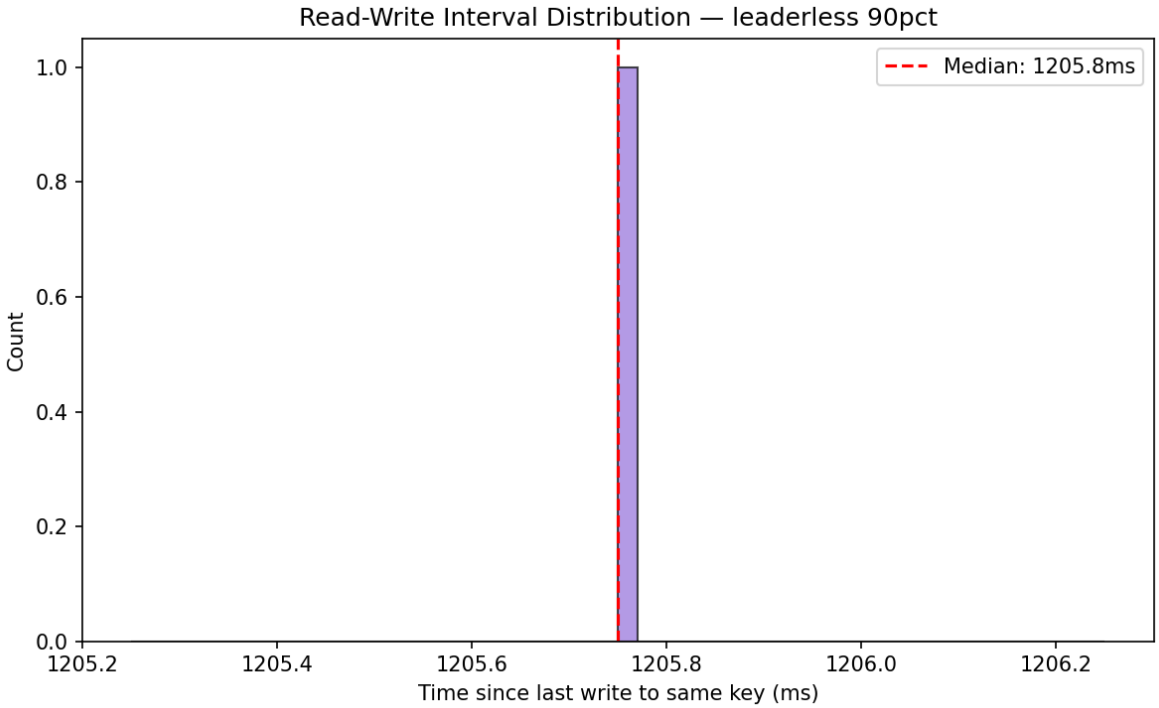
Leader-Follower W=3 R=3





Leaderless W=N R=1


Latency Distribution — leaderless 90pct
Read Latency (n=96)
Median: 1.1ms
P99: 13.1ms
Count
Latency (ms)
Write Latency (n=904)
Median: 1211.6ms
P99: 1248.9ms
Count
Latency (ms)


Read-Write Interval Distribution — leaderless 90pct
Median: 1205.8ms
Count
Time since last write to same key (ms)

Screenshot Appendix

The report package also includes screenshots of the executed tests and load-test summaries. A contact sheet is shown below; the individual PNG files remain in the screenshots directory for submission support.

AI Disclosure

AI assistance was used while developing and reviewing this assignment. The final code, tests, graphs, and report were inspected and revised so the implementation and the discussion in this report are understood and can be explained.